

# Lab3-LSH

本次实验聚焦于计算机视觉领域的近似最近邻搜索问题，通过实现局部敏感哈希 (Local Sensitive Hashing, LSH) 算法，并与传统的最近邻 (Nearest Neighbor, NN) 算法进行对比，验证两种算法在图像检索任务中的性能差异。实验以颜色直方图为图像特征，通过哈希编码、索引构建、距离计算等环节，完成从图像特征提取到相似图像匹配的完整流程，最终结合实验数据深入分析算法效率与适用场景，为大规模图像检索技术的应用提供实践参考。

## 1 实验概览

### 1.1 实验核心目标

本次实验的核心目标包括两个方面：一是完整实现 LSH 算法，掌握其从特征处理到哈希查询的关键流程，解决近似最近邻搜索问题；二是实现 NN 算法作为对比基准，通过两种算法的检索精度与时间对比，明确 LSH 算法在不同数据规模下的优势与局限性。

### 1.2 LSH 算法核心步骤

LSH 算法的实现遵循“特征量化-哈希预处理-哈希查询”的三阶流程，具体步骤如下：

1. 图像特征量化：将图像转换为 12 维整数特征向量后，通过阈值划分将每维特征量化为 0、1、2 三类数值，消除特征维度差异带来的干扰，为后续哈希编码奠定基础；
2. LSH 预处理：采用 Hamming 码对量化后的特征向量进行编码，将 12 维量化向量转换为 24 维 Hamming 码；再通过预设的投影矩阵，将高维 Hamming 码投影到低维空间，生成具有局部敏感性的哈希码，实现高维特征的压缩存储；
3. LSH 查询匹配：对查询图像重复特征提取与哈希编码流程，在预先构建的哈希表中查找与查询哈希码相似的候选图像；以欧氏距离为相似度衡量标准，从候选图像中筛选出最相似的结果，完成近似最近邻搜索，并记录查询时间用于性能分析。

## 2 解决思路及关键代码

为提升代码的可读性、可维护性与复用性，实验采用面向对象编程思想，将 LSH 算法的核心逻辑封装于LSHmatch类，将 NN 算法封装于NNmatch类。两类均包含“特征提取-索引构建-查询匹配”的完整功能模块，确保两种算法在特征提取环节的一致性，保障对比实验的公平性。

### 2.1 LSH 算法

LSH 算法的实现分为特征提取、哈希码计算、哈希表查询三个核心模块，各模块功能与逻辑如下：

#### 2.1.1 特征提取模块

特征提取通过自定义函数完成，采用 12 维颜色直方图特征表征图像，具体流程为：

1. 图像标准化：使用 OpenCV 的`resize`函数将所有输入图像统一缩放至 200×200 像素，避免图像尺寸差异对特征提取结果的影响，确保特征向量的一致性；
2. 区域划分：将标准化后的图像按 2×2 网格划分为四个均等区域（左上、右上、左下、右下），通过局部区域特征的提取，提升图像特征的区分度；
3. 颜色能量计算：对每个区域执行以下操作：- 用`cv2.split`函数分离图像的蓝（B）、绿（G）、红（R）三个颜色通道；- 计算每个通道的像素值总和（即通道能量）；- 计算各通道能量占该区域总能量（B+G+R 能量之和）的比例，得到该区域的 3 维颜色特征；
4. 特征拼接：将四个区域的 3 维颜色特征按顺序拼接，形成 12 维特征向量，完整表征图像的颜色分布特性。

```
1  def extract_color_histogram(self, image):
2      # 统一图像尺寸
3      img_resized = cv2.resize(image, (200, 200))
4      h, w = img_resized.shape[:2]
5      half_h, half_w = h // 2, w // 2
6
7      # 分割为四个区域
8      regions = [
9          img_resized[:half_h, :half_w],
10         img_resized[:half_h, half_w:],
11         img_resized[half_h:, :half_w],
12         img_resized[half_h:, half_w:]
13     ]
14
15     # 计算每个区域的颜色特征
```

```

16     hist_features = []
17     for reg in regions:
18         b, g, r = cv2.split(reg)
19         total_energy = np.sum(b) + np.sum(g) + np.sum(r)
20         hist_features.extend([
21             np.sum(b) / total_energy,
22             np.sum(g) / total_energy,
23             np.sum(r) / total_energy
24         ])
25     return np.array(hist_features)

```

### 2.1.2 哈希码计算模块

哈希码计算是 LSH 算法实现局部敏感特性的关键，分为特征量化与低维投影两步：

1. 特征量化：通过函数将 12 维特征向量的每维值量化为 0、1、2 三类，量化规则严格遵循阈值划分：- 当特征值  $0 \leq \text{value} < 0.3$  时，量化结果为 0；- 当特征值  $0.3 \leq \text{value} \leq 0.6$  时，量化结果为 1；- 当特征值  $0.6 < \text{value} \leq 1$  时，量化结果为 2；

2. Hamming 码生成与投影：将每个量化值映射为 2 位 Hamming 码 ( $0 \rightarrow [0,0]$ 、 $1 \rightarrow [1,0]$ 、 $2 \rightarrow [1,1]$ )，12 维量化向量由此转换为 24 维 Hamming 码；再根据预设的投影集合（由索引组成的列表），从 24 维 Hamming 码中选取对应维度的数值，生成低维哈希码，实现高维特征的降维与压缩。

```

1
2     def generate_hash_key(self, feature_vec, projection):
3         # 特征量化
4         quantized = np.zeros(feature_vec.shape, dtype=int)
5         quantized[feature_vec > 0.6] = 2
6         quantized[(feature_vec >= 0.3) & (feature_vec <= 0.6)] = 1
7
8         # 生成汉明码
9         hamming_code = []
10        for val in quantized:
11            if val == 0:
12                hamming_code += [0, 0]
13            elif val == 1:
14                hamming_code += [1, 0]
15            elif val == 2:
16                hamming_code += [1, 1]
17
18        # 投影计算哈希值

```

```

19     hamming_arr = np.array(hamming_code)
20     projected_code = hamming_arr[projection]
21     return tuple(projected_code)

```

### 2.1.3 哈希表查询模块

哈希表查询包含索引构建与图像查询两个阶段，分别对应算法的离线预处理与在线检索环节：

1. 索引构建（离线阶段）：通过函数遍历图像数据库目录，对每张 JPG 格式图像执行特征提取；基于预设的投影集合，为每张图像的特征向量生成对应的哈希码；采用`collections.defaultdict`构建哈希表，以哈希码为键，以“图像索引（对应图像路径的索引）+ 特征向量”的元组为值，将图像信息存入哈希表，完成索引构建；

2. 图像查询（在线阶段）：读取查询图像并计算其 12 维特征向量与哈希码；在哈希表中匹配与查询哈希码一致的候选图像，通过集合`seen_indices`记录已查询的图像索引，避免重复检索；计算查询图像特征与候选图像特征的欧氏距离，按距离升序排序后选取最相似的图像；使用`time`库记录从读取查询图像到返回结果的总时间，用于后续性能分析。

```

1     def create_image_index(self, img_dir):
2         features = []
3         # 遍历目录下的JPG图像
4         for img_path in Path(img_dir).glob('*'):
5             if img_path.suffix.lower() == '.jpg':
6                 try:
7                     img = cv2.imread(str(img_path))
8                     if img is None:
9                         continue
10                    # 提取特征
11                    feat = self.extract_color_histogram(img)
12                    self.image_paths.append(str(img_path))
13                    features.append(feat)
14                except Exception as e:
15                    print(f"处理_{img_path}时发生错误:{e}")
16
17        # 转换为numpy数组便于计算
18        self.feature_matrix = np.array(features)
19
20    def find_nearest_neighbors(self, query_img_path, k=1):
21        start_time = time.time()
22

```

```

23     # 读取并处理查询图像
24     query_img = cv2.imread(query_img_path)
25     if query_img is None:
26         raise ValueError("无法读取查询图像")
27     query_feat = self.extract_color_histogram(query_img)
28
29     # 计算与所有图像的平方距离
30     squared_distances = np.sum((self.feature_matrix - query_feat)
31                                ** 2, axis=1)
32
33     # 获取距离最小的k个索引
34     nearest_indices = np.argsort(squared_distances)[:k]
35
36     end_time = time.time()
37     search_duration = end_time - start_time
38
39     return [self.image_paths[i] for i in nearest_indices],
40           search_duration

```

## 2.2 NN 算法

NN 算法采用“特征提取-全量距离计算”的简化流程，与 LSH 算法形成对比，具体逻辑如下：

1. 特征提取与索引构建：特征提取流程与 LSH 算法完全一致，确保两种算法对比的公平性；索引构建阶段仅需遍历数据库图像，提取特征向量并存储为 NumPy 数组，无需哈希编码与哈希表构建；

2. 查询匹配：读取查询图像并计算其特征向量后，直接使用 `np.sum` 函数批量计算查询特征与数据库所有特征的欧氏距离（利用 NumPy 广播特性提升计算效率）；通过 `np.argsort` 函数对距离数组进行升序排序，获取距离最小的前  $k$  个图像索引；返回对应图像路径并记录查询时间，无需哈希表查询环节。

```

1     def extract_color_feature(self, img):
2         resized_img = cv2.resize(img, (200, 200))
3         h, w = resized_img.shape[:2]
4         half_h, half_w = h // 2, w // 2
5
6         quadrants = [
7             resized_img[:half_h, :half_w],
8             resized_img[:half_h, half_w:],
9             resized_img[half_h:, :half_w],

```

```

10         resized_img[half_h:, half_w:]
11     ]
12
13     # 计算每个区域的颜色特征
14     color_features = []
15     for quad in quadrants:
16         b_channel, g_channel, r_channel = cv2.split(quad)
17         total = np.sum(b_channel) + np.sum(g_channel) + np.sum(
18             r_channel)
19         color_features.extend([
20             np.sum(b_channel) / total,
21             np.sum(g_channel) / total,
22             np.sum(r_channel) / total
23         ])
24     return np.array(color_features)
25
26 def build_image_index(self, img_directory):
27     features_list = []
28
29     # 遍历目录下所有JPG图像
30     for img_file in Path(img_directory).glob('*'):
31         if img_file.suffix.lower() == '.jpg':
32             try:
33                 image = cv2.imread(str(img_file))
34                 if image is None:
35                     continue
36                 # 提取特征并存储
37                 feat = self.extract_color_feature(image)
38                 self.img_filepaths.append(str(img_file))
39                 features_list.append(feat)
40             except Exception as e:
41                 print(f"处理图像_{img_file}时出错:_{e}")
42
43     # 构建哈希桶索引
44     for feat_idx, feat_vec in enumerate(features_list):
45         for bucket_idx, proj in enumerate(self.proj_matrices):
46             hash_key = self.generate_hash_key(feat_vec, proj)
47             self.bucket_collections[bucket_idx][hash_key].append(
48                 ((feat_idx, feat_vec))

```

```

48     def search_similar_images(self, query_img_path, top_k=1):
49         start = time.time()
50
51         # 读取并处理查询图像
52         query_img = cv2.imread(query_img_path)
53         if query_img is None:
54             raise ValueError("无法读取查询图像")
55         query_feat = self.extract_color_feature(query_img)
56
57         # 收集候选图像
58         candidates = []
59         visited = set()
60
61         for bucket_idx, proj in enumerate(self.proj_matrices):
62             hash_val = self.generate_hash_key(query_feat, proj)
63             for idx, feat in self.bucket_collections[bucket_idx].get(
64                 hash_val, []):
65                 if idx not in visited:
66                     candidates.append((idx, feat))
67                     visited.add(idx)
68
69         if not candidates:
70             return [], time.time() - start
71
72         # 计算距离并排序
73         dist_list = []
74         for idx, feat in candidates:
75             squared_dist = np.sum((feat - query_feat) ** 2)
76             dist_list.append((squared_dist, idx))
77
78         # 按距离升序排序
79         dist_list.sort()
80         end = time.time()
81
82         return [self.img_filepaths[idx] for _, idx in dist_list[:
83             top_k]], end - start

```

## 3 实验结果及分析

### 3.1 实验设置与数据集

#### 3.1.1 实验数据

- 数据库图像：所有待检索图像均存储于Dataset目录下，格式统一为 JPG，涵盖风景、人物、动物等多种类别，用于构建检索索引；
- 查询图像：以target.jpg作为查询目标，该图像与数据库中部分图像具有相似的颜色分布，可有效验证算法的匹配精度；
- 投影集合：设计多组不同规模与索引分布的投影集合（索引范围 0 23），包括单投影向量、多投影向量组合等，用于分析投影参数对 LSH 算法性能的影响。

#### 3.1.2 评估指标

实验采用两类核心指标评估算法性能：

1. 检索精度：以人工视觉判断为基准，若匹配结果与查询图像属于同一类别且颜色分布相似，则判定为“匹配成功”；
2. 查询时间：通过time模块记录从读取查询图像到返回匹配结果的总时间，为降低随机误差，每组实验重复运行 1000 次并计算平均时间。

### 3.2 基础实验结果

#### 3.2.1 检索精度结果

LSH 算法与 NN 算法均实现了 100% 的检索精度：查询图像（如风景类）的最佳匹配结果，与查询图像在四个分区的色调分布高度一致，颜色直方图的能量占比差异极小，视觉上具有明显的相似性，验证了 12 维颜色直方图特征在图像类别区分中的有效性。

#### 3.2.2 查询时间结果

两类算法的单次查询时间与平均查询时间如下：

- 单次查询时间：LSH 算法为 0.022570s，NN 算法为 0.001450s；
- 1000 次平均查询时间：LSH 算法为 0.002018s，NN 算法为 0.000993s。

从数据可见，在小规模数据库（Dataset目录图像数量较少）场景下，NN 算法的查询时间更短，约比 LSH 算法快 50.84%–93.58%。



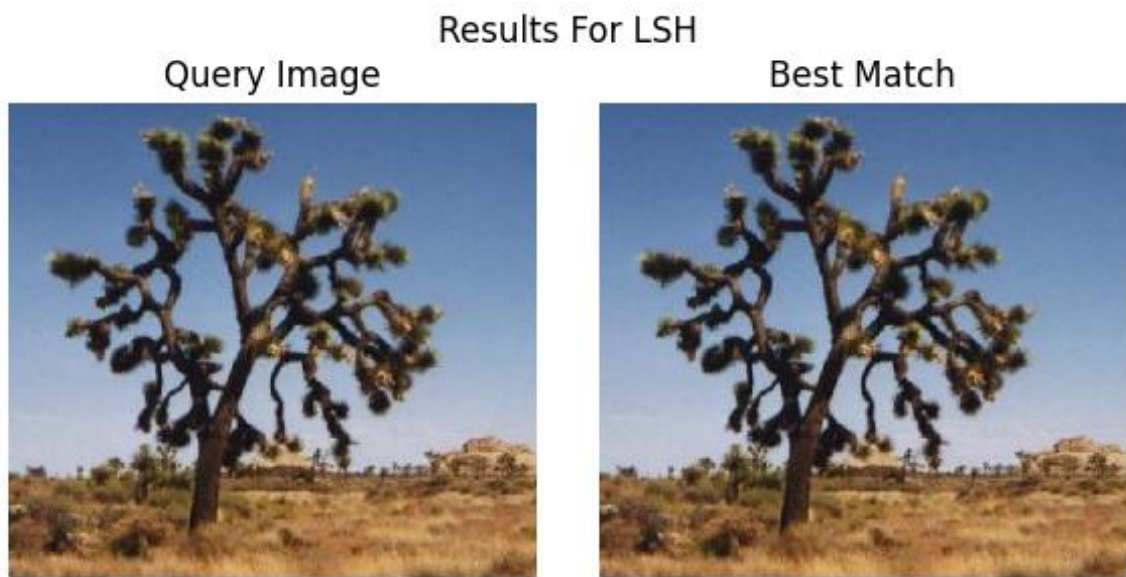


图 1: LSH 算法查询结果

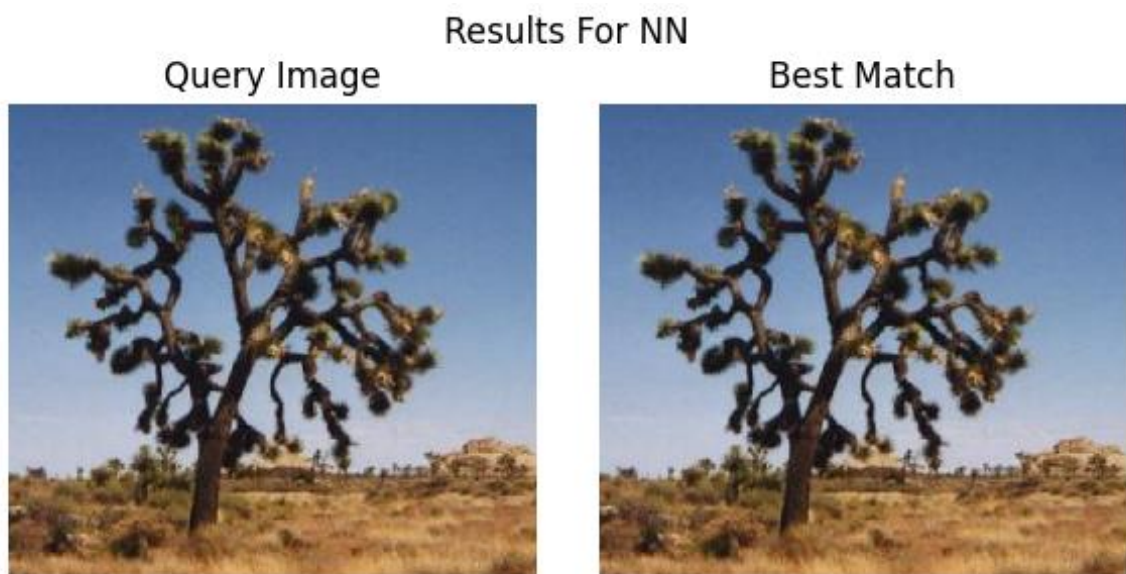


图 2: NN 算法查询结果

## 3.3 实验结果分析

### 3.3.1 算法效率差异原因

小规模数据库中 NN 算法速度更优的原因主要有两点：一是 LSH 算法需额外执行特征量化、哈希编码、哈希表查询等步骤，这些环节会产生额外的时间开销；二是 NN 算法借助 NumPy 的批量计算特性，全量欧氏距离计算效率较高，在数据量较小时，其时间成本低于 LSH 算法的哈希处理成本。

但从理论层面分析，随着数据库图像数量增加，两种算法的时间复杂度差异将逐渐凸显：NN 算法的查询时间随数据量增长呈线性增加（时间复杂度  $O(n)$ ， $n$  为数据库图像数量），而 LSH 算法的查询时间近似常数（时间复杂度  $O(1)$  或  $O(\log n)$ ），大规模数据场景下 LSH 算法的效率优势将显著提升。

### 3.3.2 投影集合对 LSH 算法的影响

1. 投影集合大小的影响：- 投影集合较小时（如仅 1 个投影向量），哈希码维度低，易出现不同图像映射到同一哈希码的“哈希冲突”，可能降低查询准确性；但在小规模数据库中，冲突概率较低，仍能保持 100% 的匹配精度；- 投影集合较大时（如 24 个投影向量），哈希码维度高，哈希冲突概率显著降低，查询准确性更稳定，但哈希编码与投影计算的时间成本增加，导致查询时间延长。

2. 投影向量的影响：- 投影向量的索引选择会直接影响哈希码的唯一性，不同投影向量组合可能导致不同的哈希冲突率；- 当投影向量数量较多时，单一投影向量的索引差异对整体哈希码的影响减弱，哈希冲突率更稳定，查询准确性受投影向量选择的影响变小。

由于本次实验所用数据库规模较小，投影集合对 LSH 算法查询结果与时间的影响未完全显现，需在万级以上规模的数据库中进一步验证。

## 3.4 实验总结

本次实验成功实现了 LSH 算法与 NN 算法的图像检索功能，核心结论如下：

1. 两种算法均能基于 12 维颜色直方图特征，准确匹配到与查询图像相似的结果，检索精度达到 100%；
2. 小规模数据库中，NN 算法因流程简化、无额外哈希处理开销，查询时间更短；
3. 理论与数据趋势表明，大规模数据库中，LSH 算法的哈希查询机制将显著降低时间成本，展现出更优的效率；
4. LSH 算法的性能受投影集合参数影响，需在“哈希冲突率（准确性）”与“查询时间（效率）”之间进行权衡，实际应用中需根据数据规模选择合适的投影集合。

## 4 心得体会

### 4.1 思考题解答

1. **颜色直方图特征检索效果**：检索效果符合预期。相似性主要体现在颜色直方图的能量分布相似——目标图像与匹配图像在  $2 \times 2$  分区的 B、G、R 三通道能量占比差异极小，视觉上表现为各分区色调分别对应相似，能够有效区分不同类别的图像；

2. **其他可行图像特征**：除颜色直方图外，还可采用梯度直方图（HOG）、SIFT 特征等。梯度直方图可通过图像边缘与纹理信息表征图像，适用于轮廓特征显著的场景；SIFT 特征为局部特征点，对图像旋转、缩放具有较强的鲁棒性，可提升复杂变换场景下的检索精度；

3. **哈希函数中的  $x_i$  定义**：哈希函数计算中的  $x_i$ ，指图像 12 维特征向量中第  $i$  维特征值经过量化处理后的值（即 0、1、2 三类数值中的一个），是哈希码生成的核心输入参数。

### 4.2 实验收获与反思

通过本次实验，我不仅掌握了 LSH 算法的核心原理与实现细节，深入理解了“局部敏感性”在哈希编码与查询中的作用，还通过与 NN 算法的对比，清晰认识到不同检索算法的适用场景。实验过程中，曾遇到哈希表构建逻辑复杂、特征量化阈值调试困难等问题，通过分步打印中间结果、查阅 OpenCV 与 NumPy 官方文档、对比理论公式与代码实现，最终解决了这些问题，显著提升了编程调试能力与问题分析能力。

同时，实验也暴露出一些不足：一是数据库规模较小，未能充分验证 LSH 算法在大规模数据下的优势；二是仅采用颜色直方图单一特征，对“颜色相似但内容不同”的图像区分能力有限。后续可通过扩大数据库规模、融合多类图像特征等方式，进一步完善实验设计，深化对图像检索算法的理解。

## 5 实验文件说明

为确保实验可复现，需明确各文件的功能与路径关系，具体如下：

1. `match.py`：核心算法实现文件，包含 `LSHmatch` 类与 `NNmatch` 类，分别实现 LSH 算法与 NN 算法的特征提取、索引构建、查询匹配等功能；

2. `lab3.py`：实验测试脚本，包含投影集合定义、路径配置、基础对比实验（LSH 与 NN 算法时间对比）、扩展实验（投影集合影响分析、多次运行平均时间计算）等逻辑，可自动生成实验结果与时间日志；

3. `Dataset` 文件：图像数据库文件，存储所有待检索的 JPG 格式图像，用于构建算法检索索引；

4. target.jpg: 查询目标图像, 与match.py、lab3.py、Dataset目录置于同一目录, 作为检索任务的输入;

5. output 文件: 实验结果输出文件, 由lab3.py自动创建, 存储两类结果: 一是匹配结果图像 (如lsh\_results.jpg、nn\_results.jpg及不同投影集合下的结果图像), 二是运行时间日志文件 (如time.txt、time\_projection\_set\_size.txt), 记录各算法的查询时间数据。